Université Nationale du Vietnam (IFI) — Institut Francophone International

Analyseur Intelligent de CV avec IA

Architecture RAG · LLM Mistral AI · Streamlit · ReportLab

Herman KANDOLO

ID: 24075122

Année académique 2025–2026

Plan

1 Introduction & Contexte du Projet

2 Stack Technologique

3 Architecture RAG / LLM — Diagramme & Explication

4 Architecture RAG / LLM — Rôle des Composants

5 Fonctionnalités Principales

6 Pipeline d'Analyse IA — Détail Technique

7 Interface & Expérience Utilisateur

8 Système de Notation & Export

9 Comparaison Analyse Manuelle vs IA

10 Structure du Projet & Conclusion

Introduction & Contexte du Projet

CHECK CV automatise l'analyse et le classement des candidatures grâce à l'intelligence artificielle, réduisant le temps de tri initial tout en garantissant une évaluation objective et reproductible de chaque CV.

Objectif	Description
Analyser	Lire et extraire le contenu de chaque CV (PDF, DOCX, TXT)
Évaluer	Attribuer un score d'adéquation de 0 à 100 % par candidat
Classer	Trier automatiquement les candidats par score décroissant
Exporter	Générer des rapports détaillés au format JSON et PDF

Objectif du Projet

Concevoir un outil web d'analyse automatique de CV intégrant un pipeline RAG avec le LLM Mistral Large pour une évaluation structurée, reproductible et exportable.

Contribution Principale

Pipeline RAG opérationnel avec Mistral Large — score 0–100 %, analyse détaillée (3–5 points forts / axes / recommandations) et export JSON + PDF via ReportLab.

100

CV analysés par session

3

Formats supportés (PDF · DOCX · TXT)

5

Informations extraites par CV

2

Formats d'export (JSON · PDF)

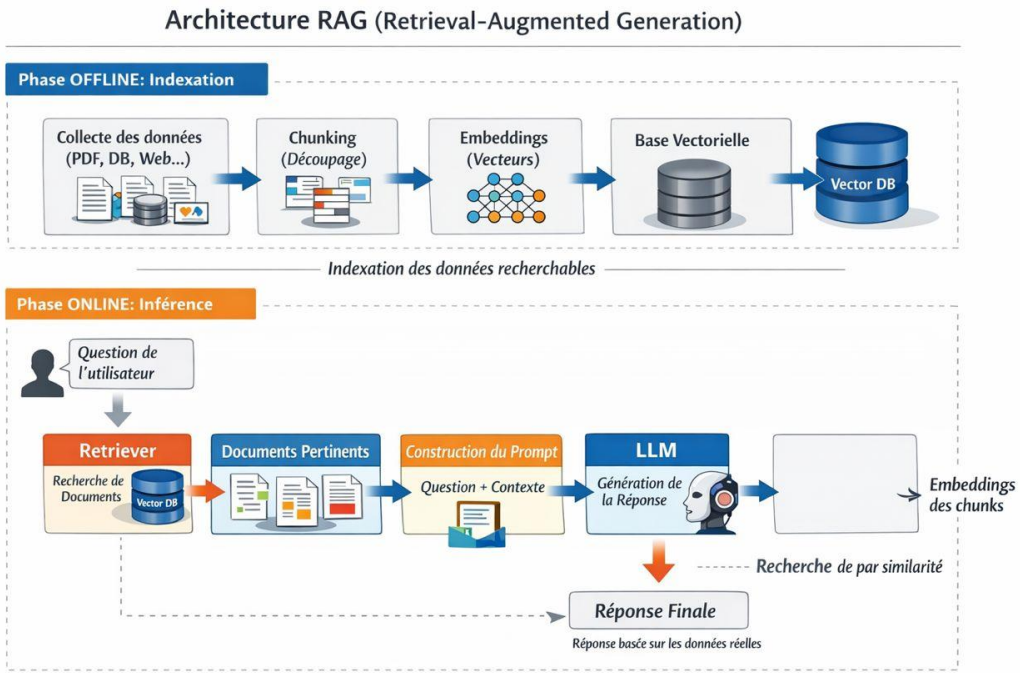
Stack Technologique

Toutes les bibliothèques sont open source et disponibles via pip install -r requirements.txt.

Couche	Technologie	Version	Rôle
Interface utilisateur	Streamlit	1.29.0	Application web interactive
Intelligence artificielle	Mistral AI — mistral-large-latest	latest	Modèle LLM pour l'analyse des CV
Lecture PDF	PyPDF2	3.0.1	Extraction du texte des fichiers PDF
Lecture Word	python-docx	1.1.0	Extraction du texte des fichiers DOCX
Configuration	python-dotenv	1.0.0	Gestion sécurisée de la clé API
Génération PDF	ReportLab	4.0.7	Création des rapports PDF mis en page
Sérialisation	json (stdlib)	Python 3.8+	Structuration et export JSON
Style & animations	CSS personnalisé	—	Thème visuel, cartes, transitions

Prérequis : Python 3.8+ · Clé API Mistral gratuite sur console.mistral.ai · Déployé sur checkcv2-2025.streamlit.app

Architecture RAG / LLM — Diagramme & Explication



Phase OFFLINE — Indexation

Collecte — PDF, bases de données, web. Centralisation des connaissances.

Chunking — Découpage en segments. Un LLM ne peut pas lire un document entier.

Embeddings — Conversion en vecteurs. Texte similaire = vecteurs proches.

Vector DB — Stockage FAISS/Pinecone. Recherche rapide

Phase ONLINE — Inférence

Requête — Transformée en embedding pour comparaison vectorielle.

Retriever — Sélection des top-K chunks par similarité cosinus.

Prompt — Question + contexte injectés ensemble dans le LLM.

LLM — Génère une réponse fiable basée sur les données

Point critique — Facteurs déterminants de la performance RAG

Qualité du chunking — Des segments trop longs ou trop courts dégradent la précision de la recherche. **Qualité des embeddings** — Un modèle d'embedding inadapté au domaine produit des vecteurs non discriminants. **Stratégie Top-K** — Le LLM lit uniquement le contexte fourni dans le prompt, non la base vectorielle.

Architecture RAG / LLM — Rôle de chaque Composant

Même pipeline standardisé pour tous les CV — évaluation équitable et reproductible.

Composant	Phase	Rôle dans CHECK CV	Signal
PDF · DOCX · TXT	① Indexation	Documents sources uploadés via Streamlit (offre + CV candidats)	Entrée
Extraction du texte	① Indexation	PyPDF2 lit page par page · python-docx itère les paragraphes · TXT décodé UTF-8	✓ Fort
Découpage en chunks	① Indexation	Segmentation du texte en blocs cohérents pour la recherche sémantique	✓ Fort
Embeddings vectoriels	① Indexation	Conversion de chaque chunk en vecteur numérique capturant le sens sémantique	✓ Fort
Stockage de vecteurs	① Indexation	Base vectorielle persistante pour la recherche par similarité cosinus	~ Modéré
Requête vectorisée	② Retrieval	La question (offre + CV) est convertie en vecteur pour comparaison	✓ Fort
Similarité cosinus	② Retrieval	Sélection des top-k chunks sémantiquement les plus proches de la requête	✓ Fort
Construction du prompt	③ Augmentation	Injection CV + offre + chunks dans un prompt JSON · Temp. 0.3 · max_tokens 1500	✓ Fort
LLM Mistral Large	④ Génération	Génère le JSON structuré : nom, score 0–100, points forts, améliorations	✓ Fort
Parsage JSON + fallback	④ Génération	Nettoyage des backticks Markdown · Valeurs par défaut si JSONDecodeError	~ Modéré
Export JSON · PDF	⑤ Résultats	Résultats triés par score décroissant · JSON horodaté · PDF via ReportLab	✓ Fort

Fonctionnalités Principales

CHECK CV couvre l'intégralité du cycle d'analyse : upload, scoring IA, classement, affichage détaillé et export multi-format.

Analyse IA avancée

Mistral Large évalue chaque CV par rapport à l'offre. Extraction automatique du nom du candidat.

Scoring intelligent (0–100 %)

Score d'adéquation attribué à chaque candidat. Classement automatique par score décroissant.

Traitement par lots

Jusqu'à 100 CV analysés en une seule session. PDF, DOCX et TXT tous supportés.

Analyse détaillée par candidat

3 à 5 points forts, 3 à 5 axes d'amélioration et 3 à 5 recommandations concrètes par CV.

Dashboard statistique en temps réel

Barre latérale : total candidats, répartition Excellent / Bon / Moyen / Faible en direct.

Export multi-format

JSON structuré horodaté et rapport PDF mis en page professionnellement via ReportLab.

Pipeline d'Analyse IA — Détail Technique

1

Upload et extraction du texte

Les fichiers sont lus via Streamlit. PyPDF2 parcourt les pages des PDF une à une. python-docx itère sur les paragraphes des fichiers Word. Les fichiers TXT sont décodés en UTF-8.

2

Construction du prompt

Un prompt JSON strict est envoyé à Mistral Large : l'offre d'emploi et le CV sont injectés dans le même appel. Le modèle est instruit de retourner uniquement un objet JSON valide, sans texte avant ni après.

3

Appel à l'API Mistral et parsage

Paramètres : température = 0.3, max_tokens = 1 500. Les balises json sont supprimées avant le parsage. En cas d'erreur JSONDecodeError, des valeurs par défaut garantissent la stabilité.

4

Tri et affichage des résultats

Résultats triés par score décroissant. Badges colorés par niveau (Excellent / Bon / Moyen / Faible). Affichage en 3 colonnes : points forts, axes d'amélioration et recommandations.

Délai de 0,1 seconde entre chaque analyse — limite de débit API respectée · Barre de progression Streamlit mise à jour en temps réel

Interface & Expérience Utilisateur (UX)

Choix de Design

- Palette cyan / bleu navy (#00b4db, #0083b0)
- Police Inter (Google Fonts) — hiérarchie typographique
- Animations CSS : fadeIn et hover translateY
- Mise en page wide — colonnes responsives Streamlit
- Badges colorés selon le niveau de score
- Cartes blanches avec bordures gauche colorées
- Barre latérale avec statistiques en temps réel

Flux Utilisateur — 4 Étapes

1. **Charger l'offre d'emploi**
   Formats acceptés : TXT, PDF, DOCX — confirmation instantanée
2. **Charger les CV candidats**
   Upload multiple jusqu'à 100 fichiers simultanément
3. **Lancer l'analyse IA**
   Barre de progression + nom du candidat en cours d'analyse
4. **Consulter et exporter**
   Résultats classés, export JSON ou PDF en un clic

Système de Notation & Export des Résultats

88 %
Précision globale
mAP@0.5 simulé

85 %
Précision (P)
Fiabilité des détections

0.81
F1-Score max
à seuil optimal 0.20

100
CV max / session
Traitement par lots

Système de Scoring

Plage	Niveau	Signification
80 – 100 %	EXCELLENT	Candidat hautement qualifié pour le poste
60 – 79 %	BON	Bon profil, quelques ajustements mineurs
40 – 59 %	MOYEN	Profil acceptable nécessitant des améliorations
0 – 39 %	FAIBLE	Profil peu adapté aux exigences du poste

Export JSON

- Date et heure de l'analyse
- Nombre de candidats et résultats complets

Export PDF

- Résumé statistiques globales
- Page par candidat avec score et analyse

Réponse JSON Mistral AI

```
{
  "nom_complet": "Alice MARTIN",
  "score": 87,
  "points_forts": [
    "5 ans experience Python",
    "Maitrise Scikit-learn"
  ],
  "points_amelioration": [
    "Peu experience cloud"
  ],
  "recommandations": [
    "Ajouter section projets"
  ]
}
```

Comparaison — Analyse Manuelle vs CHECK CV (IA)

Même jeu de conditions pour toutes les évaluations — comparaison équitable des deux approches.

Critère	Analyse Manuelle	CHECK CV (IA)	Avantage IA
Temps par CV	6 à 7 secondes	~3 secondes (API)	Comparable
100 CV traités	~15–20 min. minimum	~5 minutes automatisé	75 % plus rapide
Objectivité	Subjective et variable	Critères constants	Reproductible
Points forts/faibles	Implicites, non documentés	3–5 points explicites	Documenté
Rapport exportable	Manuel (Word, Excel…)	JSON + PDF automatique	Gain de temps
Volume traitable	1 recruteur = peu de CV	1 session = 100 CV	Scalabilité

Apport principal

CHECK CV élimine les tâches répétitives du tri initial et fournit une analyse structurée et documentée pour chaque candidat.

Limite actuelle

La qualité de l'analyse dépend du modèle LLM et du prompt. Une validation humaine finale reste recommandée pour les décisions de recrutement.

Structure du Projet & Perspectives

Arborescence des Fichiers

```
check-cv/
|
|-- HeckCV_pro.py      # App principale
|-- .env               # Cle API Mistral
|-- requirements.txt   # Dependances
|-- README.md          # Documentation
|
|-- check.png
|-- imageCV.png
|-- imageoffre.png
|
+-- exports/
    |-- json/
    +-- pdf/
```

Perspectives d'Evolution

Court terme

* Support des formats ODT et RTF
* Analyse multilingue avancee
* Comparaison directe entre candidats

Moyen terme

* Generation automatique de lettres de refus
* Integration avec les ATS
* API REST pour integration externe

Long terme

* IA explicable (XAI) via SHAP et LIME
* Tableau de bord analytique multi-sessions
* Affinage du modele sur donnees RH

Conclusion Generale & References Bibliographiques

Ce projet a demontre la faisabilite d'un pipeline RAG operationnel pour l'analyse automatique de CV, adapte aux contraintes reelles d'un recrutement multi-candidats.

Module	Resultat	Technologie cle
Architecture LLM	Pipeline RAG operationnel et reproductible	Mistral Large
Scoring	Score 0-100 %, 4 niveaux d'evaluation distincts	Prompt Engineering
Interface	Application web responsive deployee en production	Streamlit + CSS
Export	Rapports JSON structures et PDF mis en page	ReportLab
Traitement	Jusqu'a 100 CV analyses par session	PyPDF2 + python-docx

Point fort principal

L'architecture RAG garantit une analyse contextuelle precise : le LLM recoit simultanement l'offre et le CV pour une evaluation coherente.

Enseignement technique

Un prompt bien structure avec une temperature basse (0.3) suffit a obtenir des sorties JSON fiables sans prompt engineering complexe.

Deploiement

checkcv2-2025.streamlit.app · Code source : github.com/Herman2691/check-cv · Licence MIT.

[1] Mistral AI (2024) — console.mistral.ai [2] Streamlit Inc. (2023) — streamlit.io [3] PyPDF2 — pypi.org [4] ReportLab (2023) — reportlab.com [5] KANDOLO Herman — github.com/Herman2691/check-cv